Embedded OS Implementation, Spring 2025

PA1

Student ID: M11202154

Name: 呂奇

[PART I] Task Control Block Linked List

1. The screenshot results.

```
OSTick    created, Thread ID  1132
Task[ 63] created, TCB Address  b0f9c0
------After TCB[63] begin linked------
Previous TCB point to address        0
Current  TCB point to address   b0f9c0
Next     TCB point to address        0

The file 'TaskSet.txt' was opened
Task[  1] created, TCB Address  b0fa18
------After TCB[ 1] begin linked------
Previous TCB point to address        0
Current  TCB point to address   b0fa18
Next     TCB point to address   b0f9c0

Task[  2] created, TCB Address  b0fa70
------After TCB[ 2] begin linked------
Previous TCB point to address        0
Current  TCB point to address   b0fa70
Next     TCB point to address   b0fa18

================TCB linked list================
Task    Prev_TCB_addr   TCB_addr  Next_TCB_addr
 2                0       b0fa70       b0fa18
 1           b0fa70       b0fa18       b0f9c0
63           b0fa18       b0f9c0            0
```

2. Implementation

```c
        if (ptcb->OSTCBPrio == OS_TASK_IDLE_PRIO)
            printf("Task[%3.1d] created, TCB Address\t%x\n", ptcb->OSTCBPrio, ptcb);
        else
            printf("Task[%3.1d] created, TCB Address\t%x\n", ptcb->OSTCBId, ptcb);
        if (ptcb->OSTCBPrio == OS_TASK_IDLE_PRIO)
            printf("------After TCB[%2d] begin linked------\n", ptcb->OSTCBPrio);
        else
            printf("------After TCB[%2d] begin linked------\n", ptcb->OSTCBId);
        printf("Previous TCB point to address\t%6x\n", ptcb->OSTCBPrev);
        printf("Current\t TCB point to address\t%6x\n", ptcb);
        printf("Next\t TCB point to address\t%6x\n\n", ptcb->OSTCBNext);
```

Creating a task will call OSTaskCreateExt() , in which OS_TCBInit() w ill initialize the Task Control Block (TCB) and link it to the TCB ch

ain. Therefore, after the TCB is successfully created and linked, rel evant information will be printed out.

```c
printf("==============TCB linked list==============\n");
printf("Task\tPrev_TCB_addr\tTCB_addr  Next_TCB_addr\n");

OS_TCB* pCurTCB = OSTCBList;
while (pCurTCB) {
    printf("%2d\t     %6x\t  %6x       %6x\n", (pCurTCB->OSTCBPrio == OS_TASK_IDLE_PRIO) ?
        pCurTCB->OSTCBPrio : pCurTCB->OSTCBId, pCurTCB->OSTCBPrev, pCurTCB, pCurTCB->OSTCBNext);
    pCurTCB = pCurTCB->OSTCBNext;
}

printf("Tick\tEvent\t\tCurrentTask ID\tNextTask ID\tResponseTime\tPreemptionTime\tOSTimeDly\n");
```

After all tasks have been created, traverse the TCB chain to explore all tasks and print out the information.

[PART II] RM Scheduler Implementation

1. The screenshot schedule results of examples

a. Task Set 1

Tick	Event	CurrentTask ID	NextTask ID	ResponseTime	PreemptionTime	OSTimeDly
1	Completion	task(1)(0)	task(2)(0)	1	0	2
3	Preemption	task(2)(0)	task(1)(1)			
4	Completion	task(1)(1)	task(2)(0)	1	0	2
5	Completion	task(2)(0)	task(2)(1)	5	2	0
6	Preemption	task(2)(1)	task(1)(2)			
7	Completion	task(1)(2)	task(2)(1)	1	0	2
9	Completion	task(2)(1)	task(1)(3)	4	1	1
10	Completion	task(1)(3)	task(2)(2)	1	0	2
12	Preemption	task(2)(2)	task(1)(4)			
13	Completion	task(1)(4)	task(2)(2)	1	0	2
14	Completion	task(2)(2)	task(63)	4	1	1
15	Preemption	task(63)	task(1)(5)			
16	Completion	task(1)(5)	task(2)(3)	1	0	2
18	Preemption	task(2)(3)	task(1)(6)			
19	Completion	task(1)(6)	task(2)(3)	1	0	2
20	Completion	task(2)(3)	task(2)(4)	5	2	0
21	Preemption	task(2)(4)	task(1)(7)			
22	Completion	task(1)(7)	task(2)(4)	1	0	2
24	Completion	task(2)(4)	task(1)(8)	4	1	1
25	Completion	task(1)(8)	task(2)(5)	1	0	2
27	Preemption	task(2)(5)	task(1)(9)			
28	Completion	task(1)(9)	task(2)(5)	1	0	2
29	Completion	task(2)(5)	task(63)	4	1	1
30	Preemption	task(63)	task(1)(10)			

b. Task Set 2

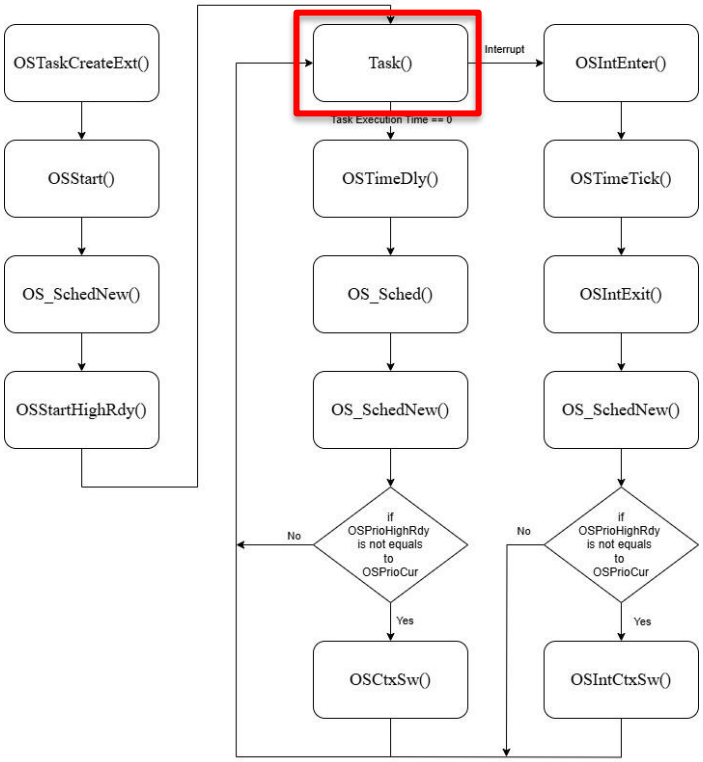
Tick	Event	CurrentTask ID	NextTask ID	ResponseTime	PreemptionTime	OSTimeDly
1	Completion	task(1)(0)	task(2)(0)	1	0	2
3	Completion	task(2)(0)	task(1)(1)	2	0	5
4	Completion	task(1)(1)	task(3)(0)	1	0	2
6	Preemption	task(3)(0)	task(1)(2)			
7	Completion	task(1)(2)	task(3)(0)	1	0	2
8	Completion	task(3)(0)	task(2)(1)	4	1	8
9	Preemption	task(2)(1)	task(1)(3)			
10	Completion	task(1)(3)	task(2)(1)	1	0	2
11	Completion	task(2)(1)	task(63)	3	1	4
12	Preemption	task(63)	task(1)(4)			
13	Completion	task(1)(4)	task(63)	1	0	2
15	Preemption	task(63)	task(1)(5)			
16	Completion	task(1)(5)	task(2)(2)	1	0	2
18	Completion	task(2)(2)	task(1)(6)	3	1	4
19	Completion	task(1)(6)	task(3)(1)	1	0	2
21	Preemption	task(3)(1)	task(1)(7)			
22	Completion	task(1)(7)	task(2)(3)	1	0	2
24	Completion	task(2)(3)	task(1)(8)	2	0	5
25	Completion	task(1)(8)	task(3)(1)	1	0	2
26	Completion	task(3)(1)	task(63)	10	7	2
27	Preemption	task(63)	task(1)(9)			
28	Completion	task(1)(9)	task(3)(2)	1	0	2
29	Preemption	task(3)(2)	task(2)(4)			
30	Preemption	task(2)(4)	task(1)(10)			

c. Task Set 3

Tick	Event	CurrentTask ID	NextTask ID	ResponseTime	PreemptionTime	OSTimeDly
1	Preemption	task(1)(0)	task(2)(0)			
3	Completion	task(2)(0)	task(1)(0)	2	0	4
5	Completion	task(1)(0)	task(3)(0)	5	2	3
7	Preemption	task(3)(0)	task(2)(1)			
9	Completion	task(2)(1)	task(1)(1)	2	0	4
12	Completion	task(1)(1)	task(3)(0)	4	1	4
13	Preemption	task(3)(0)	task(2)(2)			
15	Completion	task(2)(2)	task(3)(0)	2	0	4
15	MissDeadline	task(3)(0)	----------			

2. Implementation

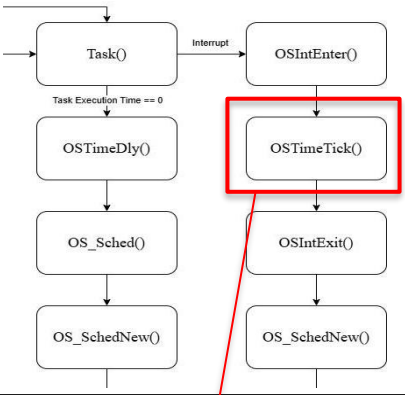
a. Task level



```c
void task(void* p_arg) {
    task_para_set* task_data;
    task_data = p_arg;
    while (1) {
        if (task_data->TaskNumber == 0 && OSTimeGet() < task_data->TaskArriveTime) {
            OSTimeDly(task_data->TaskArriveTime - OSTimeGet());
        }
        if (task_data->TaskCurrentExecutionTime == 0) {
            OSTimeDly(task_data->TaskPeriodic -
                (OSTimeGet() - (task_data->TaskNumber * task_data->TaskPeriodic + task_data->TaskArriveTime)))
        }
    }
}
```

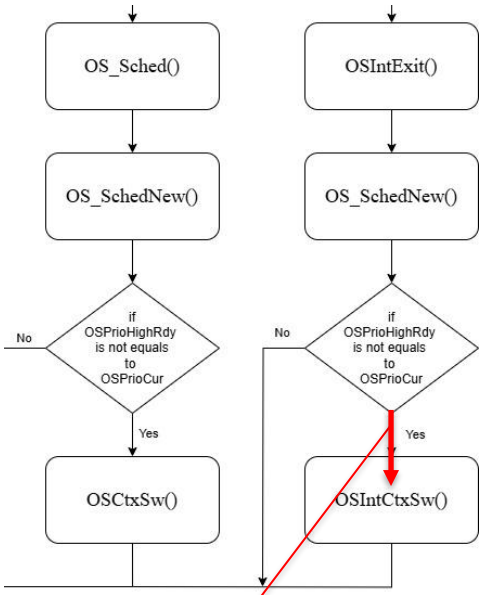
At the task level, it is necessary to check whether the arrival time needs to be delayed. When a task is completed (task current execution time is 0), it needs to enter OSTimeDly() to wait for the next task.

b. Interrupt procedure



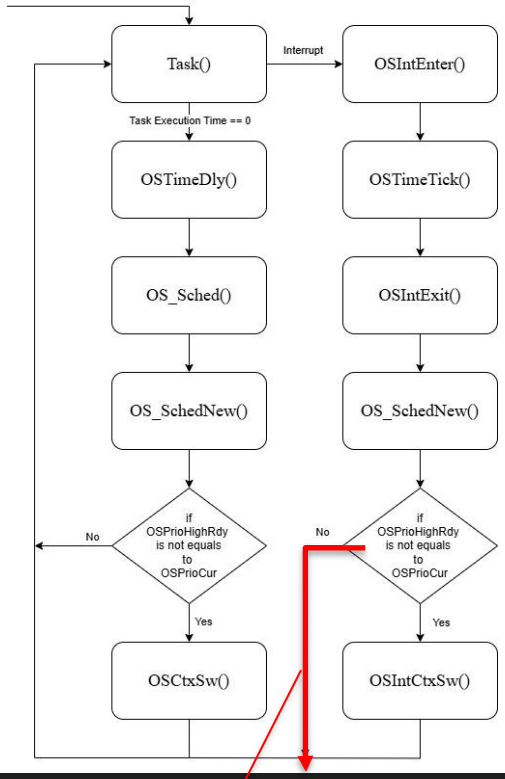

```c
#if OS_TIME_TICK_HOOK_EN > 0u
    OSTimeTickHook();                                   /* Call user definable hook                  */
#endif
#if OS_TIME_GET_SET_EN > 0u
    OS_ENTER_CRITICAL();                                /* Update the 32-bit tick counter            */
    OSTime++;
    OS_TRACE_TICK_INCREMENT(OSTime);
    OS_EXIT_CRITICAL();
#endif
    if (OSRunning == OS_TRUE) {
        /*setting the end time for the OS*/
        if (OSTimeGet() > SYSTEM_END_TIME) {
            OSRunning = OS_FALSE;
            exit(0);
        }
#if OS_TICK_STEP_EN > 0u
        switch (OSTickStepState) {                      /* Determine whether we need to process a tick  */
            case OS_TICK_STEP_DIS:                      /* Yes, stepping is disabled                    */
                step = OS_TRUE;
                break;

            case OS_TICK_STEP_WAIT:                     /* No,  waiting for uC/OS-View to set ...       */
                step = OS_FALSE;                        /*      .. OSTickStepState to OS_TICK_STEP_ONCE */
                break;

            case OS_TICK_STEP_ONCE:                     /* Yes, process tick once and wait for next ... */
                step          = OS_TRUE;                /*      ... step command from uC/OS-View        */
                OSTickStepState = OS_TICK_STEP_WAIT;
                break;

            default:                                    /* Invalid case, correct situation              */
                step          = OS_TRUE;
                OSTickStepState = OS_TICK_STEP_DIS;
                break;
        }
        if (step == OS_FALSE) {                         /* Return if waiting for step command           */
            return;
        }
#endif
        ptcb = OSTCBList;                               /* Point at first TCB in TCB list               */
        while (ptcb->OSTCBPrio != OS_TASK_IDLE_PRIO) {  /* Go through all TCBs in TCB list              */
            OS_ENTER_CRITICAL();
            if (ptcb->OSTCBDly != 0u) {                 /* Yes, Delayed or waiting for event with TO    */
                ptcb->OSTCBDly--;                       /* Decrement nbr of ticks to end of delay       */
                if (ptcb->OSTCBDly == 0u) {             /* Check for timeout                            */

                    if ((ptcb->OSTCBStat & OS_STAT_PEND_ANY)  != OS_STAT_RDY) {
                        ptcb->OSTCBStat  &= (INT8U)~(INT8U)OS_STAT_PEND_ANY;  /* Yes, Clear status flag   */
                        ptcb->OSTCBStatPend = OS_STAT_PEND_TO;                /* Indicate PEND timeout    */
                    } else {
                        ptcb->OSTCBStatPend = OS_STAT_PEND_OK;
                    }

                    if ((ptcb->OSTCBStat & OS_STAT_SUSPEND) == OS_STAT_RDY) {  /* Is task suspended?      */
                        OSRdyGrp               |= ptcb->OSTCBBitY;             /* No,  Make ready          */
                        OSRdyTbl[ptcb->OSTCBY] |= ptcb->OSTCBBitX;
                        OS_TRACE_TASK_READY(ptcb);
                    }
                }
            }
            ptcb = ptcb->OSTCBNext;                     /* Point at next TCB in TCB list                */
            OS_EXIT_CRITICAL();
        }


        TaskParameter[OSPrioCur].TaskCurrentExecutionTime--;
```

In the implementation, an interrupt is triggered every second to execute OSTimeTick(). In OSTimeTick(), the system time is incremented by one, so the current execution time of the task is decremented by one, indicating that the task has passed one time tick.




```
694  void  OSIntExit (void)
695  {
696  #if OS_CRITICAL_METHOD == 3u                          /* Allocate storage for CPU status register */
697      OS_CPU_SR  cpu_sr = 0u;
698  #endif
699      if (OSRunning == OS_TRUE) {
700          OS_ENTER_CRITICAL();
701          if (OSIntNesting > 0u) {                      /* Prevent OSIntNesting from wrapping       */
702              OSIntNesting--;
703          }
704          if (OSIntNesting == 0u) {                     /* Reschedule only if all ISRs complete ... */
705              if (OSLockNesting == 0u) {                /* ... and not locked.                      */
706                  OS_SchedNew();
707                  OSTCBHighRdy = OSTCBPrioTbl[OSPrioHighRdy];
708                  if (OSPrioCur != OS_TASK_IDLE_PRIO) {
709                      if (TaskParameter[OSPrioCur].TaskCurrentExecutionTime == 0 && OSPrioHighRdy != OSPrioCur) {    /*讓被preemp的task做完剩下的部分*/
710                          OSPrioHighRdy = OSPrioCur;
711                          OSTCBHighRdy = OSTCBPrioTbl[OSPrioCur];
712                      }
713                  }
714                  if (OSPrioHighRdy != OSPrioCur) {         /* No Ctx Sw if current task is highest rdy */
715  #if OS_TASK_PROFILE_EN > 0u
716                      if ((Output_err = fopen_s(&Output_fp, OUTPUT_FILE_NAME, "a")) == 0) {
717                          if (OSPrioCur == OS_TASK_IDLE_PRIO) {
718                              printf("%2d\tPreemption\ttask(%2d)\ttask(%2d)(%2d)\n",
719                                  OSTimeGet(), OSPrioCur, TaskParameter[OSPrioHighRdy].TaskID, TaskParameter[OSPrioHighRdy].TaskNumber);
720                              fprintf(Output_fp, "%2d\tPreemption\ttask(%2d)\ttask(%2d)(%2d)\n",
721                                  OSTimeGet(), OSPrioCur, TaskParameter[OSPrioHighRdy].TaskID, TaskParameter[OSPrioHighRdy].TaskNumber);
722                          }
723                          else {
724                              printf("%2d\tPreemption\ttask(%2d)(%2d)\ttask(%2d)(%2d)\n",
725                                  OSTimeGet(), TaskParameter[OSPrioCur].TaskID, TaskParameter[OSPrioCur].TaskNumber, TaskParameter[OSPrioHighRdy].TaskID, TaskParameter[OSPrioHighRdy].TaskNumber);
726                              fprintf(Output_fp, "%2d\tPreemption\ttask(%2d)(%2d)\ttask(%2d)(%2d)\n",
727                                  OSTimeGet(), TaskParameter[OSPrioCur].TaskID, TaskParameter[OSPrioCur].TaskNumber, TaskParameter[OSPrioHighRdy].TaskID, TaskParameter[OSPrioHighRdy].TaskNumber);
728                          }
729                          fclose(Output_fp);
730                      }
731                      OSTCBHighRdy->OSTCBCtxSwCtr++;            /* Inc. # of context switches to this task  */
732  #endif
733                      OSCtxSwCtr++;                             /* Keep track of the number of ctx switches */
734
735  #if OS_TASK_CREATE_EXT_EN > 0u
736  #if defined(OS_TLS_TBL_SIZE) && (OS_TLS_TBL_SIZE > 0u)
737                      OS_TLS_TaskSw();
738  #endif
739  #endif
740                      OS_TRACE_ISR_EXIT_TO_SCHEDULER();
741
742                      OSIntCtxSw();                         /* Perform interrupt level ctx switch       */
```

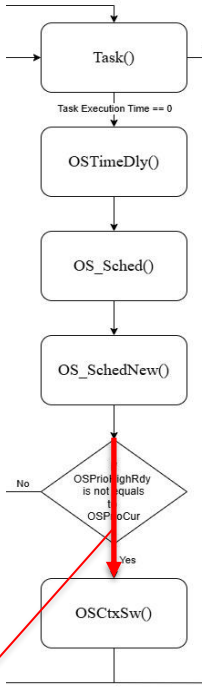
After OS_SchedNew(), the system will find the next highest-priority task. If the current task has not yet completed, it will be preempted, so preemption information will be printed at this point, and a context switch will be performed.




```
void  OSIntExit (void)
{
#if OS_CRITICAL_METHOD == 3u                          /* Allocate storage for CPU status register */
    OS_CPU_SR  cpu_sr = 0u;
#endif
    if (OSRunning == OS_TRUE) {
        OS_ENTER_CRITICAL();
        if (OSIntNesting > 0u) {                      /* Prevent OSIntNesting from wrapping       */
            OSIntNesting--;
        }
        if (OSIntNesting == 0u) {                     /* Reschedule only if all ISRs complete ... */
            if (OSLockNesting == 0u) {                /* ... and not locked.                      */
                OS_SchedNew();
                OSTCBHighRdy = OSTCBPrioTbl[OSPrioHighRdy];
                if (OSPrioCur != OS_TASK_IDLE_PRIO) {
                    if (TaskParameter[OSPrioCur].TaskCurrentExecutionTime == 0 && OSPrioHighRdy != OSPrioCur) {    /*讓被preemp的task做完剩下的部分*/
                        OSPrioHighRdy = OSPrioCur;
                        OSTCBHighRdy = OSTCBPrioTbl[OSPrioCur];
                    }
                }
                if (OSPrioHighRdy != OSPrioCur) {     /* No Ctx Sw if current task is highest rdy */
#if OS_TASK_PROFILE_EN > 0u
                    if ((Output_err = fopen_s(&Output_fp, OUTPUT_FILE_NAME, "a")) == 0) {
                        if (OSPrioCur == OS_TASK_IDLE_PRIO) {
                            printf("%2d\tPreemption\ttask(%2d)\ttask(%2d)(%2d)\n",
                                OSTimeGet(), OSPrioCur, TaskParameter[OSPrioHighRdy].TaskID, TaskParameter[OSPrioHighRdy].TaskNumber);
                            fprintf(Output_fp, "%2d\tPreemption\ttask(%2d)\ttask(%2d)(%2d)\n",
                                OSTimeGet(), OSPrioCur, TaskParameter[OSPrioHighRdy].TaskID, TaskParameter[OSPrioHighRdy].TaskNumber);
                        }
                        else {
                            printf("%2d\tPreemption\ttask(%2d)(%2d)\ttask(%2d)(%2d)\n",
                                OSTimeGet(), TaskParameter[OSPrioCur].TaskID, TaskParameter[OSPrioCur].TaskNumber, TaskParameter[OSPrioHighRdy].TaskID, TaskParameter[OSPrioHighRdy].TaskNumber);
                            fprintf(Output_fp, "%2d\tPreemption\ttask(%2d)(%2d)\ttask(%2d)(%2d)\n",
                                OSTimeGet(), TaskParameter[OSPrioCur].TaskID, TaskParameter[OSPrioCur].TaskNumber, TaskParameter[OSPrioHighRdy].TaskID, TaskParameter[OSPrioHighRdy].TaskNumber);
                        }
                        fclose(Output_fp);
                    }
                    OSTCBHighRdy->OSTCBCtxSwCtr++;        /* Inc. # of context switches to this task  */
#endif
                    OSCtxSwCtr++;                         /* Keep track of the number of ctx switches */

#if OS_TASK_CREATE_EXT_EN > 0u
#if defined(OS_TLS_TBL_SIZE) && (OS_TLS_TBL_SIZE > 0u)
                    OS_TLS_TaskSw();
#endif
#endif

                    OS_TRACE_ISR_EXIT_TO_SCHEDULER();

                    OSIntCtxSw();                         /* Perform interrupt level ctx switch       */
```

If the current task has already completed, it must wait until it calls OSTimeDly() and print completion information before a higher-priority task is scheduled for execution.

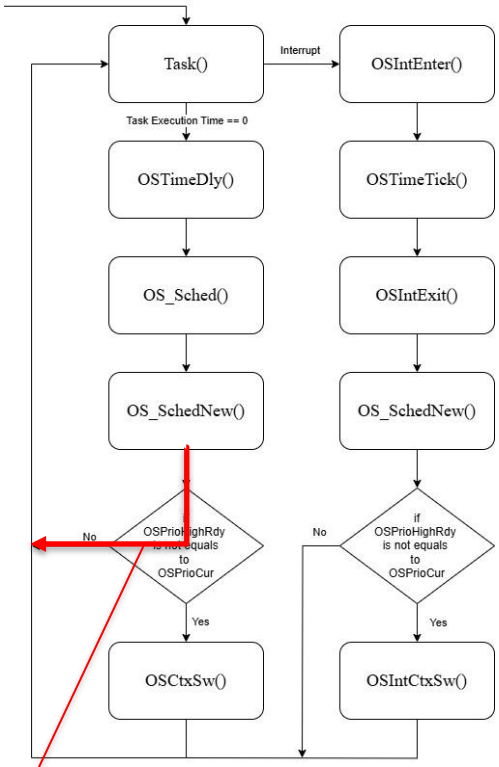
c. Task complete procedure




```
            OS_SchedNew();
            OSTCBHighRdy = OSTCBPrioTbl[OSPrioHighRdy];
            int response_time = OSTimeGet() - (TaskParameter[OSPrioCur].TaskPeriodic * TaskParameter[OSPrioCur].TaskNumber + TaskParameter[OSPrioCur].TaskArriveTime);
            int preemption_time = response_time - TaskParameter[OSPrioCur].TaskExecutionTime;
            int os_time_delay = TaskParameter[OSPrioCur].TaskPeriodic - response_time;
            if (OSPrioHighRdy != OSPrioCur) {           /* Ret ctx Sw if current task is highest rdy    */

#if OS_TASK_PROFILE_EN > 0u
                OSTCBHighRdy->OSTCBCtxSwCtr++;          /* Inc. # of context switches to this task      */
#endif
                OSCtxSwCtr++;                           /* Increment context switch counter             */

#if OS_TASK_CREATE_EXT_EN > 0u
#if defined(OS_TLS_TBL_SIZE) && (OS_TLS_TBL_SIZE > 0u)
                OS_TLS_TaskSw();
#endif
#endif

                if (TaskParameter[OSPrioCur].TaskCurrentExecutionTime == 0) {
                    if ((Output_err = fopen_s(&Output_fp, OUTPUT_FILE_NAME, "a")) == 0) {
                        if (OSPrioHighRdy == OS_TASK_IDLE_PRIO) {
                            printf("%2d\tCompletion\ttask(%2d)(%2d)\ttask(%2d)\t", OSTimeGet(), TaskParameter[OSPrioCur].TaskID, TaskParameter[OSPrioCur].TaskNumber, OSPrioHighRdy);
                            fprintf(Output_fp, "%2d\tCompletion\ttask(%2d)(%2d)\ttask(%2d)\t", OSTimeGet(), TaskParameter[OSPrioCur].TaskID, TaskParameter[OSPrioCur].TaskNumber, OSPrioHighRdy);
                        }
                        else {
                            printf("%2d\tCompletion\ttask(%2d)(%2d)\ttask(%2d)(%2d)\t", OSTimeGet(), TaskParameter[OSPrioCur].TaskID, TaskParameter[OSPrioCur].TaskNumber, TaskParameter[OSPrioHighRdy].TaskID, TaskParameter[OSPrioHighRdy].TaskNumber);
                            fprintf(Output_fp, "%2d\tCompletion\ttask(%2d)(%2d)\ttask(%2d)(%2d)\t", OSTimeGet(), TaskParameter[OSPrioCur].TaskID, TaskParameter[OSPrioCur].TaskNumber, TaskParameter[OSPrioHighRdy].TaskID, TaskParameter[OSPrioHighRdy].TaskNumber);
                        }
                        printf("      %d\t      %d\t      %d\n", response_time, preemption_time, os_time_delay);
                        fprintf(Output_fp, "      %d\t      %d\t      %d\n", response_time, preemption_time, os_time_delay);
                        TaskParameter[OSPrioCur].TaskNumber++;
                        TaskParameter[OSPrioCur].TaskCurrentExecutionTime = TaskParameter[OSPrioCur].TaskExecutionTime;
                        fclose(Output_fp);
                    }
```

When a task's current execution time reaches 0, it will proactively call OSTimeDly(). If the task needs to be awakened after a certain period, the system will select the highest-priority task for execution, print the completion information, increment the task number, and reset the current execution time. Finally, a context switch will be performed to allow the high-priority task to execute.

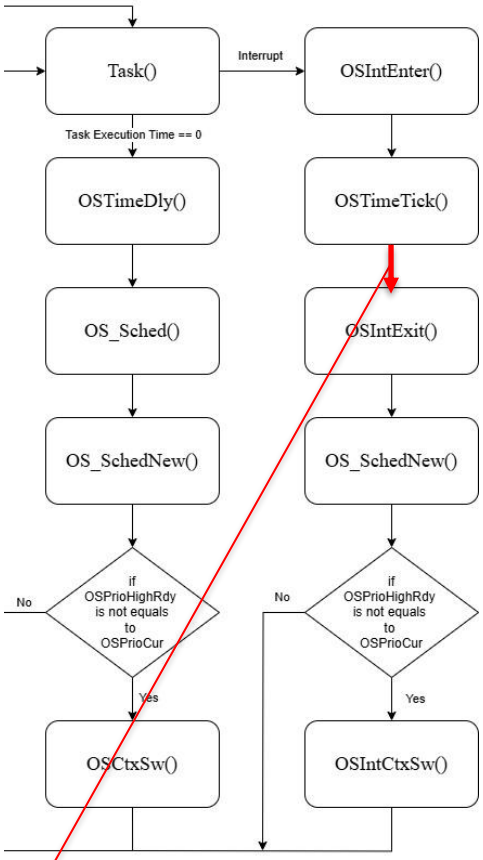
Response time is calculated as the difference between the current time and the task's arrival time.
Preemption time is calculated as the difference between the response time and the execution time.
OSTimeDly is calculated as the difference between the task period and the response time.




```
else if (ticks == 0u && TaskParameter[OSPrioCur].TaskCurrentExecutionTime==0) {
    int response_time = OSTimeGet() - (TaskParameter[OSPrioCur].TaskPeriodic * TaskParameter[OSPrioCur].TaskNumber + TaskParameter[OSPrioCur].TaskArriveTime);
    int preemption_time = response_time - TaskParameter[OSPrioCur].TaskExecutionTime;
    int os_time_delay = TaskParameter[OSPrioCur].TaskPeriodic - response_time;
    if ((Output_err = fopen_s(&Output_fp, OUTPUT_FILE_NAME, "a")) == 0) {
        printf("%2d\tCompletion\ttask(%2d)(%2d)\ttask(%2d)(%2d)\t", OSTimeGet(), TaskParameter[OSPrioCur].TaskID, TaskParameter[OSPrioCur].TaskNumber, TaskParameter[OSPrioHighRdy].TaskID, TaskParameter[OSPrioHighRdy].TaskNumber + 1);
        fprintf(Output_fp, "%2d\tCompletion\ttask(%2d)(%2d)\ttask(%2d)(%2d)\t", OSTimeGet(), TaskParameter[OSPrioCur].TaskID, TaskParameter[OSPrioCur].TaskNumber, TaskParameter[OSPrioHighRdy].TaskID, TaskParameter[OSPrioHighRdy].TaskNumber + 1);
        printf("      %d\t      %d\t      %d\n", response_time, preemption_time, os_time_delay);
        fprintf(Output_fp, "      %d\t      %d\t      %d\n", response_time, preemption_time, os_time_delay);
        fclose(Output_fp);
    }
    if (missdeadline != TASK_NUMBER + 1) {
        if ((Output_err = fopen_s(&Output_fp, OUTPUT_FILE_NAME, "a")) == 0) {
            printf("%2d\tMissDeadline\ttask(%2d)(%2d)\t-----------------\n", OSTimeGet(), TaskParameter[missdeadline].TaskID, TaskParameter[missdeadline].TaskNumber);
            fprintf(Output_fp, "%2d\tMissDeadline\ttask(%2d)(%2d)\t-----------------\n", OSTimeGet(), TaskParameter[missdeadline].TaskID, TaskParameter[missdeadline].TaskNumber);
            fclose(Output_fp);
        }
        OSRunning = OS_FALSE;
        exit(0);
    }
    TaskParameter[OSPrioCur].TaskNumber++;
    TaskParameter[OSPrioCur].TaskCurrentExecutionTime = TaskParameter[OSPrioCur].TaskExecutionTime;
}
}
```

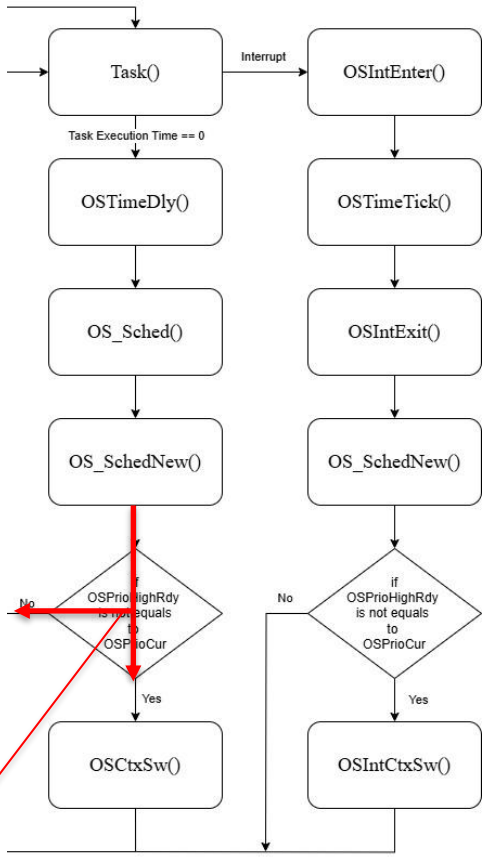
When the current task completes and the same task continues, no delay is needed. Therefore, it only needs to print the completion information, increment the task number, and reset the current execution time.

3. Handle the deadline missing situation




```
for (int i = 0; i < TASK_NUMBER; i++) {
    if ((OSTimeGet() + TaskParameter[i].TaskPeriodic - TaskParameter[i].TaskArriveTime) % TaskParameter[i].TaskPeriodic == 0 &&
        (OSTimeGet()-TaskParameter[i].TaskArriveTime)/TaskParameter[i].TaskPeriodic > TaskParameter[i].TaskNumber) {
        if (TaskParameter[i].TaskCurrentExecutionTime != 0) {
            if (TaskParameter[OSPrioCur].TaskCurrentExecutionTime == 0) {
                missdeadline = i;
            }
            else {
                if ((Output_err = fopen_s(&Output_fp, OUTPUT_FILE_NAME, "a")) == 0) {
                    printf("%2d\tMissDeadline\ttask(%2d)(%2d)\t------------------\n", OSTimeGet(), TaskParameter[i].TaskID, TaskParameter[i].TaskNumber);
                    fprintf(Output_fp, "%2d\tMissDeadline\ttask(%2d)(%2d)\t------------------\n", OSTimeGet(), TaskParameter[i].TaskID, TaskParameter[i].TaskNumber);
                    fclose(Output_fp);
                }
                OSRunning = OS_FALSE;
                exit(0);
            }
        }
    }
}
```

At each time tick, it is necessary to check if any task has missed it s deadline. First, identify which tasks have reached their next perio d. Then, compare the expected number of completions in a normal scena rio with the current number of completions. If the current completion count has not reached the expected count, the task is considered to h ave missed its deadline.




```
if (missdeadline != TASK_NUMBER + 1) {
    if ((Output_err = fopen_s(&Output_fp, OUTPUT_FILE_NAME, "a")) == 0) {
        printf("%2d\tMissDeadline\ttask(%2d)(%2d)\t------------------\n", OSTimeGet(), TaskParameter[missdeadline].TaskID, TaskParameter[missdeadline].TaskNumber);
        fprintf(Output_fp, "%2d\tMissDeadline\ttask(%2d)(%2d)\t------------------\n", OSTimeGet(), TaskParameter[missdeadline].TaskID, TaskParameter[missdeadline].TaskNumber);
        fclose(Output_fp);
    }
    OSRunning = OS_FALSE;
    exit(0);
}
```

If the current task has completed, the completion information needs to be printed first. Therefore, after the time tick check, the system should record which task has missed its deadline. Once the completion information has been printed, the missed deadline information should then be printed.

[PART III] FIFO Scheduler Implementation

1. The screenshot schedule results of examples

   a. TaskSet 1

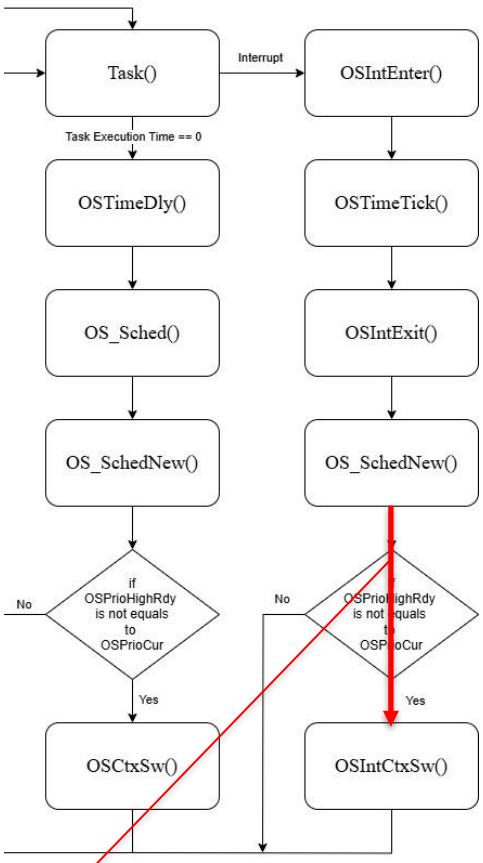
Tick	Event	CurrentTask ID	NextTask ID	ResponseTime	PreemptionTime	OSTimeDly
1	Completion	task(1)(0)	task(2)(0)	1	0	3
4	Completion	task(2)(0)	task(1)(1)	4	1	1
5	Completion	task(1)(1)	task(2)(1)	1	0	3
8	Completion	task(2)(1)	task(1)(2)	3	0	2
9	Completion	task(1)(2)	task(63)	1	0	3
10	Preemption	task(63)	task(2)(2)			
13	Completion	task(2)(2)	task(1)(3)	3	0	2
14	Completion	task(1)(3)	task(63)	2	1	2
15	Preemption	task(63)	task(2)(3)			
18	Completion	task(2)(3)	task(1)(4)	3	0	2
19	Completion	task(1)(4)	task(63)	3	2	1
20	Preemption	task(63)	task(1)(5)			
21	Completion	task(1)(5)	task(2)(4)	1	0	3
24	Completion	task(2)(4)	task(1)(6)	4	1	1
25	Completion	task(1)(6)	task(2)(5)	1	0	3
28	Completion	task(2)(5)	task(1)(7)	3	0	2
29	Completion	task(1)(7)	task(63)	1	0	3
30	Preemption	task(63)	task(2)(6)			

b. TaskSet 2

Tick	Event	CurrentTask ID	NextTask ID	ResponseTime	PreemptionTime	OSTimeDly
1	Completion	task(1)(0)	task(2)(0)	1	0	2
3	Completion	task(2)(0)	task(1)(1)	2	0	5
4	Completion	task(1)(1)	task(3)(0)	1	0	2
7	Completion	task(3)(0)	task(1)(2)	3	0	9
8	Completion	task(1)(2)	task(2)(1)	2	1	1
10	Completion	task(2)(1)	task(1)(3)	2	0	5
11	Completion	task(1)(3)	task(63)	2	1	1
12	Preemption	task(63)	task(1)(4)			
13	Completion	task(1)(4)	task(63)	1	0	2
15	Preemption	task(63)	task(1)(5)			
16	Completion	task(1)(5)	task(2)(2)	1	0	2
18	Completion	task(2)(2)	task(3)(1)	3	1	4
21	Completion	task(3)(1)	task(1)(6)	5	2	7
21	MissDeadline	task(1)(6)	----------			

2. Implementation

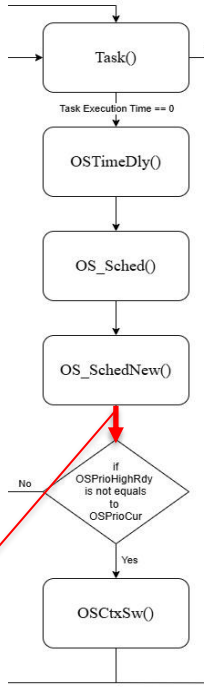
a. Interrupt procedure




```
OS_SchedNew();
OSTCBHighRdy = OSTCBPrioTbl[OSPrioHighRdy];
if (OSPrioCur != OS_TASK_IDLE_PRIO) {
    if (TaskParameter[OSPrioCur].TaskCurrentExecutionTime != 0) {
        OSPrioHighRdy = OSPrioCur;
        OSTCBHighRdy = OSTCBPrioTbl[OSPrioCur];
    }
    if (TaskParameter[OSPrioCur].TaskCurrentExecutionTime == 0 && OSPrioHighRdy != OSPrioCur) {    /*讓被preemp的task做完剩下的部分*/
        OSPrioHighRdy = OSPrioCur;
        OSTCBHighRdy = OSTCBPrioTbl[OSPrioCur];
    }
}
if (OSPrioHighRdy != OSPrioCur) {          /* No Ctx Sw if current task is highest rdy */
```

After OSIntExit() and executing OS_SchedNew(), determine whether the current task has completed. If it has not yet completed, continue executing the task without any preemption. When the current task is completed, it should proceed to execute OSTimeDly(), print the completion information, and then switch to the next task.

b. Task complete procedure




```
while (ptcb->OSTCBPrio != OS_TASK_IDLE_PRIO) {
    int next_arrive_time = TaskParameter[ptcb->OSTCBPrio].TaskPeriodic * TaskParameter[ptcb->OSTCBPrio].TaskNumber + TaskParameter[ptcb->OSTCBPrio].TaskArriveTime;
    if (ptcb->OSTCBDly == 0) {
        if (next_arrive_time < min_task_time && TaskParameter[ptcb->OSTCBPrio].TaskID < min_task_ID) {
            OSPrioHighRdy = ptcb->OSTCBPrio;
            min_task_ID = TaskParameter[ptcb->OSTCBPrio].TaskID;
            min_task_time = next_arrive_time;
        }
    }
    ptcb = ptcb->OSTCBNext;
}
```

When switching tasks, check the delay of each task. If the delay time is 0, it means the task is ready for execution. Then, compare the arrival times of all tasks the smaller the arrival time, the earlier it arrived and should be executed first. If multiple tasks have the same arrival time, the task with the smaller task ID should be executed first.

3. Comparison of RM and FIFO

The difference between RM and FIFO scheduling is that RM prioritizes tasks with shorter periods, while FIFO executes tasks in the order they arrive. In RM scheduling, tasks with longer periods may be preempted by tasks with shorter periods. In contrast, FIFO strictly follows the arrival order, preventing preemption issues. However, both scheduling methods can still lead to missed deadlines.